Drzewo BST

Generated by Doxygen 1.18.0

1 Drzewo BST w C++ 1
1.1 O projekcie . . . 1
1.2 Funkcjonalności . . . 1
1.3 Struktura plików . . . 2
1.4 Kompilacja i uruchomienie . . . 2
1.4.1 W środowisku Visual Studio . . . 2
1.4.2 Z użyciem kompilatora g++ (w konsoli) . . . 2

2 Data Structure Index 3
2.1 Data Structures . . . 3

3 File Index 5
3.1 File List . . . 5

4 Data Structure Documentation 7
4.1 BST Class Reference . . . 7
4.1.1 Detailed Description . . . 7
4.1.2 Constructor & Destructor Documentation . . . 8
4.1.2.1 BST() . . . 8
4.1.2.2 ∼BST() . . . 8
4.1.3 Member Function Documentation . . . 8
4.1.3.1 clear() . . . 8
4.1.3.2 display() . . . 8
4.1.3.3 getRoot() . . . 8
4.1.3.4 insert() . . . 9
4.1.3.5 remove() . . . 9
4.1.3.6 saveToTextFile() . . . 9
4.1.3.7 searchPath() . . . 9
4.1.3.8 setRoot() . . . 10
4.2 FileManager Class Reference . . . 10
4.2.1 Detailed Description . . . 10
4.2.2 Member Function Documentation . . . 11
4.2.2.1 loadBinary() . . . 11
4.2.2.2 loadFromText() . . . 11
4.2.2.3 saveBinary() . . . 11
4.3 Node Struct Reference . . . 12
4.3.1 Detailed Description . . . 12
4.3.2 Constructor & Destructor Documentation . . . 12
4.3.2.1 Node() . . . 12
4.3.3 Field Documentation . . . 12
4.3.3.1 data . . . 12
4.3.3.2 left . . . 12
4.3.3.3 right . . . 12

5 File Documentation **13**

5.1 BST.cpp File Reference 13

5.1.1 Detailed Description 13

5.2 BST.h File Reference 13

5.2.1 Detailed Description 13

5.3 BST.h 14

5.4 FileManager.cpp File Reference 14

5.4.1 Detailed Description 14

5.5 FileManager.h File Reference 15

5.5.1 Detailed Description 15

5.6 FileManager.h 15

5.7 main.cpp File Reference 15

5.7.1 Detailed Description 16

5.7.2 Function Documentation 16

5.7.2.1 main() 16

5.7.2.2 printMenu() 16

5.8 README.md File Reference 16

Index **17**

Chapter 1

Drzewo BST w C++

1.1 O projekcie

Projekt przedstawia implementację Drzewa Poszukiwań Binarnych (BST - Binary Search Tree) w języku C++. Program działa w oparciu o interaktywne menu konsolowe i został zaprojektowany z wykorzystaniem programowania obiektowego. Ponadto, do obsługi wyszukiwania ścieżki w drzewie wykorzystano kontener z biblioteki STL (`std::vector`).

Projekt został zrealizowany jako zadanie grupowe, którego głównym celem było praktyczne zapoznanie się z możliwościami platformy GitHub podczas pracy zespołowej (tworzenie gałęzi, praca równoległa, scalanie, celowe generowanie i rozwiązywanie konfliktów).

1.2 Funkcjonalności

- **Zarządzanie elementami:**
  - Dodawanie nowego elementu do drzewa.
  - Usuwanie pojedynczego, wybranego węzła.
  - Usuwanie (czyszczenie) całego drzewa.
- **Przeszukiwanie i wyświetlanie:**
  - Wyszukiwanie ścieżki od korzenia do zadanego elementu.
  - Wyświetlanie drzewa graficznie na ekranie trzema metodami: `Preorder`, `Inorder`, `Postorder`.
- **Obsługa plików (klasa FileManager):**
  - Zapis wygenerowanego drzewa do pliku tekstowego.
  - Zapis utworzonego drzewa do pliku w formacie binarnym.
  - Odczyt drzewa z pliku binarnego.
  - Wczytywanie liczb z pliku tekstowego, umożliwiające budowę drzewa od zera lub dodawanie do istniejącego.

1.3 Struktura plików

Projekt został podzielony na osobne pliki zgodnie z dobrymi praktykami C++:

- `main.cpp` – plik główny zawierający logikę menu i sterowanie programem.
- `BST.h` / `BST.cpp` – deklaracja i implementacja struktury węzła (`Node`) oraz samej klasy drzewa.
- `FileManager.h` / `FileManager.cpp` – deklaracja i implementacja klasy dedykowanej do operacji wejścia/wyjścia na plikach txt i binarnych.

1.4 Kompilacja i uruchomienie

1.4.1 W środowisku Visual Studio

1. Utwórz **Pusty projekt** (C++).
2. Dodaj pliki źródłowe (`main.cpp`, `BST.cpp`, `FileManager.cpp`) i nagłówkowe (`BST.h`, `FileManager.h`) do projektu (Opcja: *Dodaj -> Istniejący element*).
3. Skompiluj i uruchom za pomocą skrótu `Ctrl + F5`.

1.4.2 Z użyciem kompilatora g++ (w konsoli)

Będąc w folderze z plikami projektu, wpisz polecenie: ```bash g++ main.cpp BST.cpp FileManager.cpp -o drzewo↩
_bst

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

BST
: Klasa implementuj¹ca binarne drzewo poszukiwañ (Binary Search Tree) 7

FileManager
: Klasa zajmuj¹ca siê zapisem i odczytem struktury drzewa do/z plików binarnych i tekstowych . 10

Node
: Struktura reprezentuj¹ca pojedynczy wêze^3 w drzewie BST 12

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

BST.cpp
: Implementacja metod klasy binarnego drzewa poszukiwañ (BST) 13

BST.h
: Definicja struktury wêz^{3}a oraz klasy binarnego drzewa poszukiwañ (BST) 13

FileManager.cpp
: Implementacja metod klasy FileManager do obs^{3}ugi plików 14

FileManager.h
: Definicja klasy FileManager do zarz¹dzania operacjami wejcia/wyjcia 15

main.cpp
: GÅÃ^{3}wny plik programu realizujÄcy pÄtlÄ menu oraz interakcjÄ z uÅ¼ytkownikiem 15

Chapter 4

Data Structure Documentation

4.1 BST Class Reference

Klasa implementuj¹ca binarne drzewo poszukiwañ (Binary Search Tree).

`#include <BST.h>`

Public Member Functions

- BST ()

  Konstruktor domylny, inicjalizuje puste drzewo.
- ~BST ()

  Destruktor, czyci pamiêæ zajmowan¹ przez drzewo.
- void insert (int data)

  Dodaje now¹ wartoæ do drzewa.
- void remove (int data)

  Usuwa podan¹ wartoæ z drzewa, jeli istnieje.
- void clear ()

  Usuwa wszystkie wêz^{3}*y i zwalnia pamiêæ (czyszczenie drzewa).*
- void searchPath (int data)

  Szuka i wywietla cie¿kê do podanego elementu.
- void display (int method)

  Wywietla wartoci drzewa na ekranie wybran¹ metod¹.
- void saveToTextFile (const std::string &filename)

  Zapisuje uk^{3}*ad drzewa (preorder) do pliku tekstowego.*
- Node * getRoot () const

  Zwraca wskanik na korzeñ (u¿ywane przez FileManager).
- void setRoot (Node *newRoot)

  Ustawia nowy korzeñ drzewa (u¿ywane przy wczytywaniu z pliku bin).

4.1.1 Detailed Description

Klasa implementuj¹ca binarne drzewo poszukiwañ (Binary Search Tree).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 BST()

```
BST::BST ()
```

Konstruktor domylny, inicjalizuje puste drzewo.

4.1.2.2 ∼BST()

```
BST::∼BST ()
```

Destruktor, czyci pamiêæ zajmowan¹ przez drzewo.

4.1.3 Member Function Documentation

4.1.3.1 clear()

```
void BST::clear ()
```

Usuwa wszystkie wêz^{3}y i zwalnia pamiêæ (czyszczenie drzewa).

4.1.3.2 display()

```
void BST::display (
            int method)
```

Wywietla wartoci drzewa na ekranie wybran¹ metod¹.

Parameters

method	1 - Preorder, 2 - Inorder, 3 - Postorder.

4.1.3.3 getRoot()

```
Node * BST::getRoot () const  [inline]
```

Zwraca wskanik na korzeñ (u¿ywane przez FileManager).

Returns

Wskanik na obiekt root.

4.1.3.4 insert()

```
void BST::insert (
            int data)
```

Dodaje now¹ wartoæ do drzewa.

Parameters

data	Wartoæ ca[3]kowita do dodania.

4.1.3.5 remove()

```
void BST::remove (
            int data)
```

Usuwa podan¹ wartoæ z drzewa, jeli istnieje.

Parameters

data	Wartoæ do usuniêcia.

4.1.3.6 saveToTextFile()

```
void BST::saveToTextFile (
            const std::string & filename)
```

Zapisuje uk[3]ad drzewa (preorder) do pliku tekstowego.

Parameters

filename	Nazwa pliku wyjciowego.

4.1.3.7 searchPath()

```
void BST::searchPath (
            int data)
```

Szuka i wywietla cie¿kê do podanego elementu.

Parameters

data	Wartoæ docelowa do znalezienia.

4.1.3.8 setRoot()

```
void BST::setRoot (
            Node * newRoot)  [inline]
```

Ustawia nowy korzeñ drzewa (u¿ywane przy wczytywaniu z pliku bin).

Parameters

newRoot	Wskanik na nowy korzeñ.

The documentation for this class was generated from the following files:

- BST.h
- BST.cpp

4.2 FileManager Class Reference

Klasa zajmuj¹ca siê zapisem i odczytem struktury drzewa do/z plików binarnych i tekstowych.

```
#include <FileManager.h>
```

Public Member Functions

- void saveBinary (BST &tree, const std::string &filename)

  Zapisuje strukturê drzewa do pliku binarnego.
- void loadBinary (BST &tree, const std::string &filename)

  Wczytuje strukturê drzewa z pliku binarnego, zastêpuj¹c aktualne drzewo.
- void loadFromText (BST &tree, const std::string &filename)

  Wczytuje ci¹g liczb z pliku tekstowego, wstawiaj¹c je po kolei do drzewa.

4.2.1 Detailed Description

Klasa zajmuj¹ca siê zapisem i odczytem struktury drzewa do/z plików binarnych i tekstowych.

4.2.2 Member Function Documentation

4.2.2.1 loadBinary()

```
void FileManager::loadBinary (
            BST & tree,
            const std::string & filename)
```

Wczytuje strukturê drzewa z pliku binarnego, zastêpuj¹c aktualne drzewo.

Parameters

tree	Referencja do drzewa, do którego wczytamy dane.
filename	Nazwa pliku binarnego.

4.2.2.2 loadFromText()

```
void FileManager::loadFromText (
            BST & tree,
            const std::string & filename)
```

Wczytuje ci¹g liczb z pliku tekstowego, wstawiaj¹c je po kolei do drzewa.

Parameters

tree	Referencja do drzewa.
filename	Nazwa pliku tekstowego.

4.2.2.3 saveBinary()

```
void FileManager::saveBinary (
            BST & tree,
            const std::string & filename)
```

Zapisuje strukturê drzewa do pliku binarnego.

Parameters

tree	Referencja do obiektu drzewa, które chcemy zapisaæ.
filename	Nazwa pliku binarnego.

The documentation for this class was generated from the following files:

- FileManager.h
- FileManager.cpp

4.3 Node Struct Reference

Struktura reprezentuj¹ca pojedynczy wêze^3 w drzewie BST.

`#include <BST.h>`

Public Member Functions

- Node (int val)

  Konstruktor tworz¹cy nowy wêze^3.

Data Fields

- int data
- Node * left
- Node * right

4.3.1 Detailed Description

Struktura reprezentuj¹ca pojedynczy wêze^3 w drzewie BST.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 Node()

```
Node::Node (
            int val)  [inline]
```

Konstruktor tworz¹cy nowy wêze^3.

Parameters

val	Wartoæ do przypisania nowemu wêz^{3}owi.

4.3.3 Field Documentation

4.3.3.1 data

```
int Node::data
```

Wartoæ przechowywana w wêle

4.3.3.2 left

```
Node* Node::left
```

Wskanik na lewe poddrzewo (wartoci mniejsze)

4.3.3.3 right

```
Node* Node::right
```

Wskanik na prawe poddrzewo (wartoci wiêksze)

The documentation for this struct was generated from the following file:

- BST.h

Chapter 5

File Documentation

5.1 BST.cpp File Reference

Implementacja metod klasy binarnego drzewa poszukiwañ (BST).

```
#include "BST.h"
```

5.1.1 Detailed Description

Implementacja metod klasy binarnego drzewa poszukiwañ (BST).

5.2 BST.h File Reference

Definicja struktury wêz^{3}a oraz klasy binarnego drzewa poszukiwañ (BST).

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
```

Data Structures

- struct Node

  Struktura reprezentuj'ca pojedynczy wêze^3 w drzewie BST.
- class BST

  Klasa implementuj'ca binarne drzewo poszukiwañ (Binary Search Tree).

5.2.1 Detailed Description

Definicja struktury wêz^{3}a oraz klasy binarnego drzewa poszukiwañ (BST).

Author

Damian Kie^{3}basa, Patryk Gargula

5.3 BST.h

Go to the documentation of this file.

```
00001
00006
00007 #ifndef BST_H
00008 #define BST_H
00009
00010 #include <iostream>
00011 #include <vector>
00012 #include <fstream>
00013 #include <string>
00014
00019 struct Node {
00020     int data;
00021     Node* left;
00022     Node* right;
00023
00028     Node(int val) : data(val), left(nullptr), right(nullptr) {}
00029 };
00030
00035 class BST {
00036 private:
00037     Node* root;
00038
00039     Node* insertRec(Node* node, int data);
00040     Node* deleteRec(Node* node, int data);
00041     Node* minValueNode(Node* node);
00042     void destroyTree(Node* node);
00043     bool searchPathRec(Node* node, int data, std::vector<int>& path);
00044     void saveToTextFileRec(Node* node, std::ofstream& file);
00045
00046     void printPreorder(Node* node);
00047     void printInorder(Node* node);
00048     void printPostorder(Node* node);
00049
00050 public:
00054     BST();
00055
00059     ~BST();
00060
00065     void insert(int data);
00066
00071     void remove(int data);
00072
00076     void clear();
00077
00082     void searchPath(int data);
00083
00088     void display(int method);
00089
00094     void saveToTextFile(const std::string& filename);
00095
00100     Node* getRoot() const { return root; }
00101
00106     void setRoot(Node* newRoot) { root = newRoot; }
00107 };
00108
00109 #endif
```

5.4 FileManager.cpp File Reference

Implementacja metod klasy FileManager do obs^{3}ugi plików.

```
#include "FileManager.h"
#include <fstream>
#include <iostream>
```

5.4.1 Detailed Description

Implementacja metod klasy FileManager do obs^{3}ugi plików.

5.5 FileManager.h File Reference

Definicja klasy FileManager do zarz¹dzania operacjami wejcia/wyjcia.

```
#include "BST.h"
#include <string>
```

Data Structures

- class FileManager

  Klasa zajmuj¹ca siê zapisem i odczytem struktury drzewa do/z plików binarnych i tekstowych.

5.5.1 Detailed Description

Definicja klasy FileManager do zarz¹dzania operacjami wejcia/wyjcia.

Author

Patryk Gargula

5.6 FileManager.h

Go to the documentation of this file.

```
00001
00006
00007 #ifndef FILEMANAGER_H
00008 #define FILEMANAGER_H
00009
00010 #include "BST.h"
00011 #include <string>
00012
00017 class FileManager {
00018 private:
00019     void saveBinaryRec(Node* node, std::ofstream& file);
00020     Node* loadBinaryRec(std::ifstream& file);
00021
00022 public:
00028     void saveBinary(BST& tree, const std::string& filename);
00029
00035     void loadBinary(BST& tree, const std::string& filename);
00036
00042     void loadFromText(BST& tree, const std::string& filename);
00043 };
00044
00045 #endif
```

5.7 main.cpp File Reference

Główny plik programu realizujący pętlę menu oraz interakcję z użytkownikiem.

```
#include "BST.h"
#include "FileManager.h"
#include <iostream>
#include <string>
```

Functions

- void printMenu ()

  Wypisuje dostępne opcje menu na standardowe wyjście (konsolę).
- int main ()

  Główna funkcja programu, uruchamia pętlę obsługującą wybory użytkownika.

5.7.1 Detailed Description

Główny plik programu realizujący pętlę menu oraz interakcję z użytkownikiem.

Author

Damian Kiełbasa, Patryk Gargula

5.7.2 Function Documentation

5.7.2.1 main()

```
int main ()
```

Główna funkcja programu, uruchamia pętlę obsługującą wybory użytkownika.

Returns

0 w przypadku pomyślnego zakończenia.

5.7.2.2 printMenu()

```
void printMenu ()
```

Wypisuje dostępne opcje menu na standardowe wyjście (konsolę).

5.8 README.md File Reference

Index

~BST
    BST, 8

BST, 7
    ~BST, 8
    BST, 8
    clear, 8
    display, 8
    getRoot, 8
    insert, 8
    remove, 9
    saveToTextFile, 9
    searchPath, 9
    setRoot, 9
BST.cpp, 13
BST.h, 13

clear
    BST, 8

data
    Node, 12
display
    BST, 8
Drzewo BST w C++, 1

FileManager, 10
    loadBinary, 11
    loadFromText, 11
    saveBinary, 11
FileManager.cpp, 14
FileManager.h, 15

getRoot
    BST, 8

insert
    BST, 8

left
    Node, 12
loadBinary
    FileManager, 11
loadFromText
    FileManager, 11

main
    main.cpp, 16
main.cpp, 15
    main, 16
    printMenu, 16

Node, 12
    data, 12
    left, 12
    Node, 12
    right, 12

printMenu
    main.cpp, 16

README.md, 16
remove
    BST, 9
right
    Node, 12

saveBinary
    FileManager, 11
saveToTextFile
    BST, 9
searchPath
    BST, 9
setRoot
    BST, 9